PART III · 35 MINUTES

What a self-improving language agent learns

What should change after deployment, where should the change live, which signals justify it and when should the agent create a new learning problem?

Learning: task, performance, experience

> “A computer program is said to learn from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in T, as measured by P, improves with experience E.”
>
> Mitchell, *Machine Learning*, 1997

Example for this Tutorial:

Task:

Extract a set of relations from documents

Performance:

Relation-level precision and recall

Experience:

Documents, corrections, feedback

Time-stamp and persistent state

$$(y_t, h_t) = \text{Execute}(S_t, x_t, z_t)$$

S_t persistent state available before time-stamp t

x_t, z_t the task input, and the environment including external tools

y_t, h_t output and trace; the trace may support an update

A Post-Deployment Self-Improving (PDSI) Agent should be able to retrieve an instance of the task from memory (persistence) and also solve new tasks (learning/generalization)

The update mechanism

$$S_{t+1} = U(S_t, E_t)$$

The update procedure **U** may **propose**, **reject** or **commit** a change, or confirm current knowledge (no change). Its inputs: traces, external corrections, earlier state.

Persistence is not permanence (a memory can expire and still matter). Influence can be second-order: a revised test changes future learning decisions, not today's answers.

Important Elements

Learning mechanism: The procedure. It remains a learning algorithm even when a particular run fails.

Demonstrated improvement: An empirically supported outcome on declared targets.

Candidate learning update: An experience-driven proposed change (which may be harmful).

Adaptation: Behaviour changes in response to circumstances (transient or retained).

Self-Refine: revises an output within one interaction (adaptation without retention in Madaan et al., 2023)

Reflexion: retains verbal feedback for later trials (a different retention boundary in Shinn et al., 2023)

Important Elements: Self-Refine

Madaan, Aman, et al. "Self-refine: Iterative refinement with self-feedback." *Advances in neural information processing systems* 36 (2023): 46534-46594.

SELF-REFINE: Iterative Refinement with Self-Feedback

Aman Madaan[1], **Niket Tandon**[2], **Prakhar Gupta**[1], **Skyler Hallinan**[3], **Luyu Gao**[1], **Sarah Wiegreffe**[2], **Uri Alon**[1]*, **Nouha Dziri**[2], **Shrimai Prabhumoye**[4], **Yiming Yang**[1], **Shashank Gupta**[2], **Bodhisattwa Prasad Majumder**[5], **Katherine Hermann**[6], **Sean Welleck**[2,3], **Amir Yazdanbakhsh**[6], **Peter Clark**[2]
[1]Language Technologies Institute, Carnegie Mellon University
[2]Allen Institute for Artificial Intelligence
[3]University of Washington [4]NVIDIA [5]UC San Diego [6]Google Deepmind
selfrefine@googlegroups.com

Abstract

Like humans, large language models (LLMs) do not always generate the best output on their first try. Motivated by how humans refine their written text, we introduce SELF-REFINE, an approach for improving initial outputs from LLMs through iterative feedback and refinement. The main idea is to generate an initial output using an LLM; then, the same LLM provides *feedback* for its output and uses it to *refine* itself, iteratively. SELF-REFINE does not require any supervised training data, additional training, or reinforcement learning, and instead uses a single LLM as the generator, refiner, and feedback provider. We evaluate SELF-REFINE across 7 diverse tasks, ranging from dialog response generation to mathematical reasoning, using state-of-the-art (GPT-3.5 and GPT-4) LLMs. Across all evaluated tasks, outputs generated with SELF-REFINE are preferred by humans and automatic metrics over those generated with the same LLM using conventional one-step generation, improving by ~20% absolute on average in task performance. Our work demonstrates that even state-of-the-art LLMs like GPT-4 can be further improved at test-time using our simple, standalone approach.[2].

Important Elements: Self-Refine

Madaan, Aman, et al. "Self-refine: Iterative refinement with self-feedback." *Advances in neural information processing systems* 36 (2023): 46534-46594.

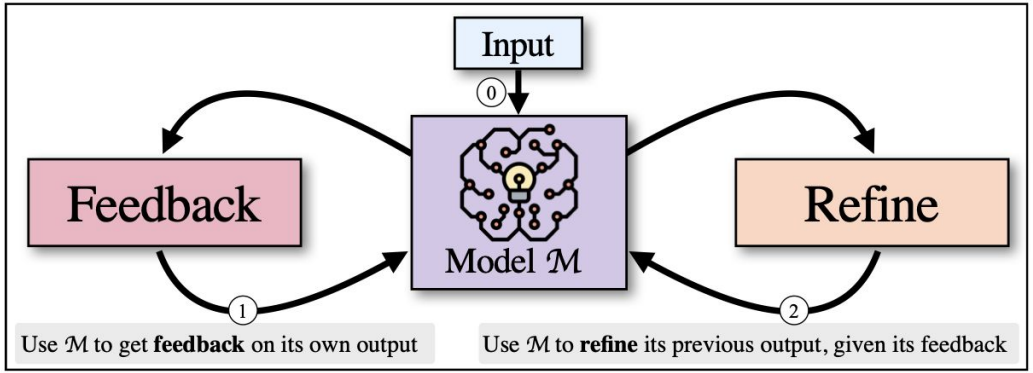


Figure 1: Given an input (⓪), SELF-REFINE starts by generating an output and passing it back to the same model $\mathcal{M}$ to get feedback (①). The feedback is passed back to $\mathcal{M}$, which refines the previously generated output (②). Steps (①) and (②) iterate until a stopping condition is met. SELF-REFINE is instantiated with a language model such as GPT-3.5 and does not involve human assistance.

Important Elements: Self-Refine

Madaan, Aman, et al. "Self-refine: Iterative refinement with self-feedback." *Advances in neural information processing systems* 36 (2023): 46534-46594.

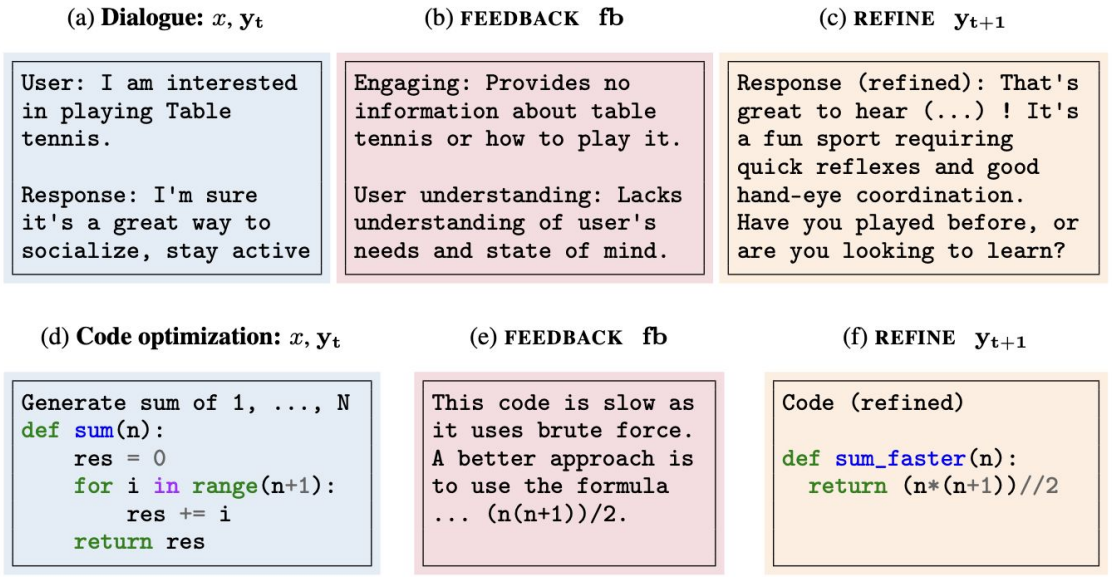


Figure 2: Examples of SELF-REFINE: an initial output generated by the base LLM and then passed back to the *same* LLM to receive feedback to the *same* LLM to refine the output . The top row illustrates this for dialog generation where an initial dialogue response can be transformed into a more engaging one that also understands the user by applying feedback. The bottom row illustrates this for code optimization where the code is made more efficient by applying feedback.

Important Elements: Reflexion

Shinn, Noah, et al. "Reflexion: Language agents with verbal reinforcement learning." *Advances in neural information processing systems* 36 (2023): 8634-8652.

Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn
Northeastern University
noahshinn024@gmail.com

Federico Cassano
Northeastern University
cassano.f@northeastern.edu

Ashwin Gopinath
MIT
agopi@mit.edu

Karthik Narasimhan
Princeton University
karthikn@princeton.edu

Shunyu Yao
Princeton University
shunyuy@princeton.edu

Abstract

Large language models (LLMs) have been increasingly used to interact with external environments (e.g., games, compilers, APIs) as goal-driven agents. However, it remains challenging for these language agents to quickly and efficiently learn from trial-and-error as traditional reinforcement learning methods require extensive training samples and expensive model fine-tuning. We propose *Reflexion*, a novel framework to reinforce language agents not by updating weights, but instead through linguistic feedback. Concretely, Reflexion agents verbally reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer to induce better decision-making in subsequent trials. Reflexion is flexible enough to incorporate various types (scalar values or free-form language) and sources (external or internally simulated) of feedback signals, and obtains significant improvements over a baseline agent across diverse tasks (sequential decision-making, coding, language reasoning). For example, Reflexion achieves a 91% pass@1 accuracy on the HumanEval coding benchmark, surpassing the previous state-of-the-art GPT-4 that achieves 80%. We also conduct ablation and analysis studies using different feedback signals, feedback incorporation methods, and agent types, and provide insights into how they affect performance. We release all code, demos, and datasets at https://github.com/noahshinn024/reflexion

Important Elements: Reflexion

Shinn, Noah, et al. "Reflexion: Language agents with verbal reinforcement learning." *Advances in neural information processing systems* 36 (2023): 8634-8652.

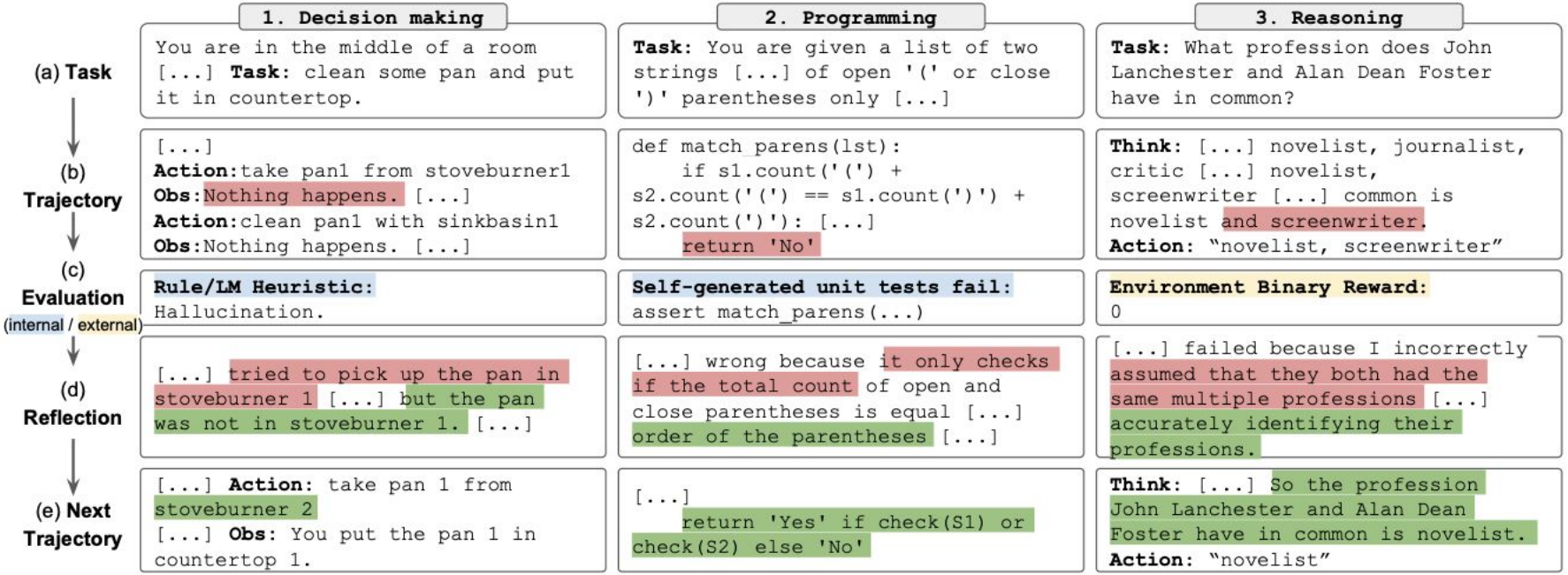
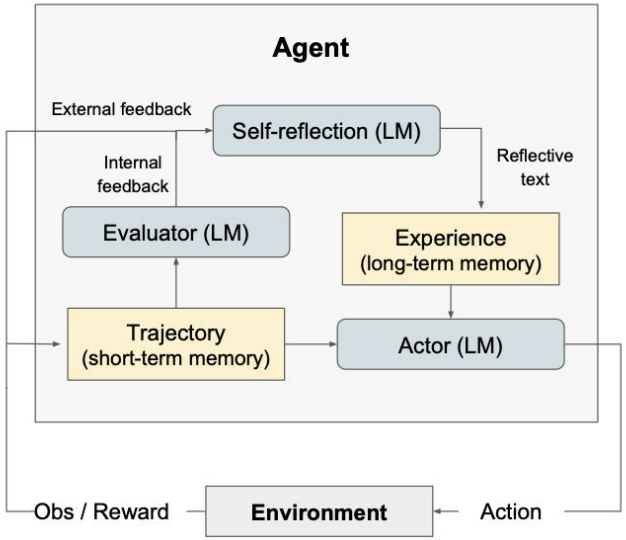


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.

Important Elements: Reflexion

Shinn, Noah, et al. "Reflexion: Language agents with verbal reinforcement learning." *Advances in neural information processing systems* 36 (2023): 8634-8652.



Algorithm 1 Reinforcement via self-reflection

Initialize Actor, Evaluator, Self-Reflection: M_a, M_e, M_{sr}
Initialize policy $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$
Generate initial trajectory using π_θ
Evaluate τ_0 using M_e
Generate initial self-reflection sr_0 using M_{sr}
Set $mem \leftarrow [sr_0]$
Set $t = 0$
while M_e not pass or $t <$ max trials **do**
    Generate $\tau_t = [a_0, o_0, \ldots a_i, o_i]$ using π_θ
    Evaluate τ_t using M_e
    Generate self-reflection sr_t using M_{sr}
    Append sr_t to mem
    Increment t
end while
return

Figure 2: (a) Diagram of Reflexion. (b) Reflexion reinforcement algorithm

Improvement is multidimensional and constrained

Measures that need not move together

Recall · precision · calibration · provenance completeness · latency · correction burden.

Accept everything: recall rises. Abstain on everything hard: precision rises.

Preservation counts

If terminology shifts, holding performance steady while a frozen baseline deteriorates is a real benefit.

Knowledge Lifecycle

When a deployed system acquires new knowledge:

- Should it remain in context or episodic memory?
- Should it become explicit in a KG/KB?
- Should it be incorporated in the Harness (tools, skills, external data, etc)
- Should it eventually be internalized in model parameters?
- How should knowledge move between representations?

Things a deployed agent can learn

Knowledge: Facts, identities, relations, constraints with scope and provenance

Memory: Episodic traces and semantic summaries of experience

Skills: Tool-use procedures, workflows, decomposition strategies

Policies: When to retrieve, ask, abstain, escalate or propose an update

Self-monitoring: Uncertainty, failure prediction, anomaly detection, diagnosis

Learning problems: New tasks, their specifications and their couplings

The categories overlap

Every learned piece of knowledge should carry its status

Learned knowledge	New fact, skill, parameters, etc.
Evidence span	The exact elements supporting the new knowledge
Source and time	Document ID, model ID, skill ID, publication time, acquisition time
Scope	Domain, model version, temporal qualifiers
Epistemic status	candidate · accepted · disputed · derived · retired

Memory systems make different design choices

Episodic — particular events with their context

Semantic — generalizations across events

System	Central mechanism	Reference
MemGPT	Moves information between context and external storage	Packer et al., 2023
A-MEM	Linked notes whose organization evolves	Xu et al., 2025
Mem0	Extraction, consolidation and retrieval of salient facts	Chhikara et al., 2025
MemInsight	Autonomous augmentation of memory representations	Salama et al., 2025
EP+SEM_CRIT	Episodic plus Semantic memory	Hassle et al., 2026
LLM-as-RNN	a frozen LLM keeps its state as natural-language memory	Lu et al., AACL2026
AutoMEM	a harness where the agent manages its own text-file memory	Chen et al., AACL2026

A memory is a typed, scoped object. It should not be just a stored chat history

Each memory should record

- what happened, and in which task
- who or what produced it
- local or global scope
- whether it may affect future answers
- evidence links and confidence
- when it expires

Typing makes memory **queryable and governable**: a RejectedClaim must never be retrieved as support; a UserPreference must not leak across users.

Skills: reusable procedures for a class of outcomes

Toolformer: Self-supervised data construction and filtering teach a model when to call APIs. Schick et al., 2023

Voyager: Stores executable skills in a library for later reuse. Wang et al., 2023

SkillWeaver: Web agents self-improve by discovering and honing skills. Zheng et al., 2025

Policies: when to retrieve, ask, abstain or update

Policy improvement depends on costs and consequences.

Always asking an expert breaks a review budget. Always abstaining destroys coverage. Evaluate routing by the **complete target outcome**: latency, review effort, routing errors.

Policies change the future data stream. If an Agent only reviews cases it already suspects, it may never discover a confidently wrong region.

Step-level refinement — learning from process-level feedback · Xiong et al., 2024
RAGEN — self-evolution via multi-turn RL · Wang et al., 2025
MemRL — learned utility of episodic memories, LM fixed · Zhang et al., 2026

Learning when not to trust yourself

SelfCheckGPT — zero-resource, black-box hallucination detection · Manakul et al., 2023

Uncertainty-aware language agents · Han et al., 2024

Uncertainty propagation across agent steps · Zhao et al., 2025

Confidence Reasoning Graphs - King et al., 2026

A confident explanation of an error is not evidence that the diagnosis is correct. Monitoring is itself a learnable task with its own failure modes.

Where learning signals come from

User feedback: Corrections, ratings, preferences — selected, not representative

Retrieval evidence: Documents that support or contradict an output

Tool traces: Errors, timeouts, schema mismatches, outputs

Environment: Observed outcomes of the agent's actions

Offline eval logs: Regression suites, audits, judge verdicts

The system's own knowledge can label data too

Ratner et al., 2017 (Snorkel)

Training on self-produced data can be biased Shumailov et al., 2024

Signals coming from the same source should not double-counted

Manage tasks as a portfolio: create, merge, retire

Proliferation: Each task adds labels, tests, storage, inference cost and dependencies. Judge marginal target value against maintenance cost.
Merging: Compare inputs, outputs, error patterns and downstream uses. Renaming two tasks into one does not make their labels compatible.
Retirement: Unhelpful, obsolete, absorbed, or its domain is gone. Retiring the learner need not delete its evidence.

Task registry keeps rationale, versions and rejected tasks with the conditions of rejection.

Self-improvement is a portfolio of update targets, each with its own substrate, signals and evidence.

A new learning task is a resource-consuming hypothesis. It earns its place through target benefit, not novelty, autonomy or auxiliary accuracy.

After the break · Part IV — design patterns that make this loop safe.